SZABIST
University

Shaheed Zulfikar Ali Bhutto
Institute of Science and Technology

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks:_____

Obtained Marks: _____

Computing Vision

Lab Task # 09

Submitted To: **Mr. Muhammad Azhar**

Student Name: **Ayesha Munir**

Reg. Number: **23108106**

Code Example 1: Extract and Compare Color Histograms

A digital art museum is building an online platform that allows users to search for artworks based on visual similarity. Since many older paintings are not tagged or described in detail, traditional keyword-based searches are ineffective. The goal is to implement a CBIR system that retrieves similar artworks from a digital collection based on image content like color and texture.

Objective:

To develop a CBIR system using histogram-based feature comparison that:

□ Extracts color histograms from images.

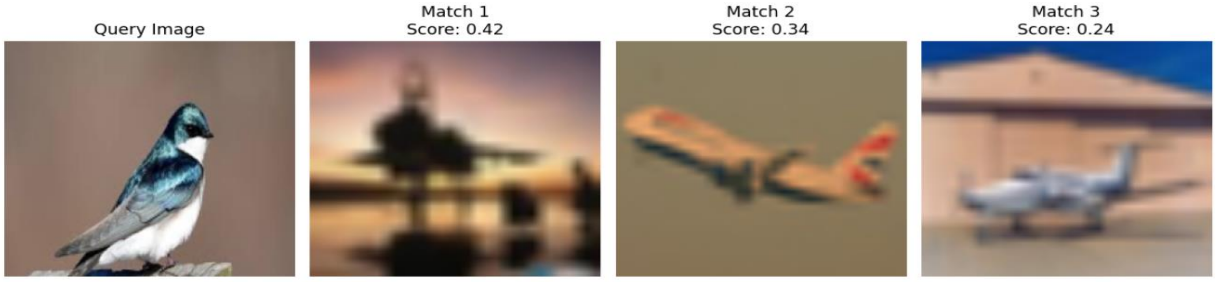
□ Compares a query image against a dataset.

□ Returns top visually similar matches using histogram correlation.

```python
# Code Example 1: Extract and Compare Color Histograms
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
dataset_path = "D:/23108106/CIFAR-10/cifar10/test"
query_path = "D:/23108106/A32.jpg"
def get_histogram(img):
    hist = cv2.calcHist([img], [0, 1, 2], None, [8, 8, 8], [0, 256, 0, 256, 0, 256])
    cv2.normalize(hist, hist)
    return hist.flatten()
query = cv2.imread(query_path)
if query is None:
    print("Error: Query image not found!")
query = cv2.resize(query, (200, 200))
query_hist = get_histogram(query)
scores = []
max_images = 150
count = 0
for root, dirs, files in os.walk(dataset_path):
    for file in files:
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            image_path = os.path.join(root, file)
            image = cv2.imread(image_path)
            if image is None:
                continue
            image = cv2.resize(image, (200, 200))
            hist = get_histogram(image)
            score = cv2.compareHist(query_hist, hist, cv2.HISTCMP_CORREL)
            scores.append((score, file, image))
            count += 1
```

```python
            if count >= max_images:
                break
    if count >= max_images:
        break
scores.sort(reverse=True)
top_matches = scores[:3]
print("Top 3 Matching Images:")
plt.figure(figsize=(12, 6))
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(query, cv2.COLOR_BGR2RGB))
plt.title("Query Image")
plt.axis('off')
for i, (score, fname, img) in enumerate(top_matches):
    plt.subplot(1, 4, i + 2)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(f"Match {i+1}\nScore: {score:.2f}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

```
Top 3 Matching Images:
```



Code Example 2: Wildlife Image Retrieval using Color-Based Content Analysis

A wildlife research center is maintaining a large collection of tiger images captured using camera traps placed in various forest reserves. These images come from different angles, lighting conditions, and environments. Researchers often need to find visually similar images of tigers (e.g., from the same region or individual tiger) for behavioral or movement pattern studies. Manual searching is time-consuming and error-prone.

Objective:

To automate the retrieval of visually similar tiger images using a Content-Based Image Retrieval (CBIR) system that leverages color histogram features in the HSV color space and cosine similarity.

Task:

Implement a Python program using OpenCV and scikit-learn to:

- Extract color histogram features from a query tiger image and a dataset of tiger-related images.
- Compare their visual similarity using cosine similarity.
- Display and rank the top 3 most similar images from the dataset.

```python
# Code Example 2: Wildlife Image Retrieval using Color-Based Content Analysis
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity
def extract_color_histogram(image, bins=(8, 8, 8)):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    hist = cv2.calcHist([hsv], [0, 1, 2], None, bins, [0,180, 0,256, 0,256])
    return hist.flatten()
img = cv2.imread("D:/23108106/A28.jpg")
query_features = extract_color_histogram(img)
dataset = []
dataset.append(cv2.imread("D:/23108106/A29.jpg"))
dataset.append(cv2.imread("D:/23108106/A30.jpg"))
dataset.append(cv2.imread("D:/23108106/A31.jpg"))
dataset_features = [extract_color_histogram(image) for image in dataset]
similarities = [cosine_similarity([query_features], [features])[0][0] for features in dataset_features]
ranked_indices = sorted(range(len(similarities)), key=lambda i: similarities[i], reverse=True)
plt.figure()
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Query Image')
plt.axis('off')
plt.show()
top_k = 3
```

```python
top_k = 3
for i in range(top_k):
    similar_img = dataset[ranked_indices[i]]
    plt.figure()
    plt.imshow(cv2.cvtColor(similar_img, cv2.COLOR_BGR2RGB))
    plt.title(f"Similar Image {i+1}")
    plt.axis('off')
    plt.show()
```

Query Image

Similar Image 1

Code Example 3: Texture-Based Defect Detection in Fabric Inspection

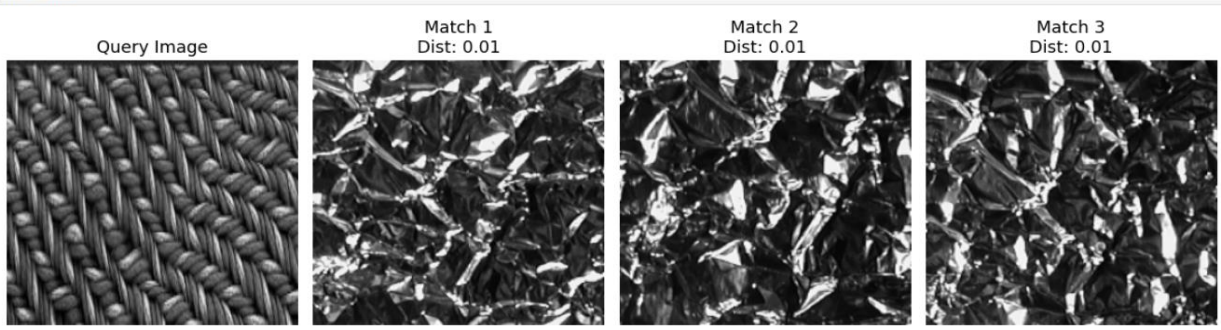
A textile manufacturing company needs to automate its quality inspection process. Traditional color-based image retrieval fails to identify subtle texture defects (like missing threads, uneven weaving, or stitching errors). Hence, the company seeks a solution based on texture similarity, not just color.

Objective:

Implement a Content-Based Image Retrieval (CBIR) system using Local Binary Pattern (LBP) to retrieve similar fabric textures from a dataset based on a given query image.

```python
# Code Example 3: Texture-Based Defect Detection in Fabric Inspection
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import local_binary_pattern
import os
dataset_path = "D:/23108106/KTH TIPS-2/KTH_TIPS"
query_img_path = "D:/23108106/A24.jpg"
def extract_lbp_features(gray_img):
    lbp = local_binary_pattern(gray_img, P=24, R=3, method="uniform")
    hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, 27), range=(0, 26))
    hist = hist.astype("float")
    hist /= (hist.sum() + 1e-6)
    return hist
query_img = cv2.imread(query_img_path, cv2.IMREAD_GRAYSCALE)
query_img = cv2.resize(query_img, (256, 256))
query_hist = extract_lbp_features(query_img)
def chi2_distance(histA, histB, eps=1e-6):
    return 0.5 * np.sum(((histA - histB) ** 2) / (histA + histB + eps))
scores = []
for root, dirs, files in os.walk(dataset_path):
    for file in files:
        if file.lower().endswith('.jpg', '.jpeg', '.png'):
            img_path = os.path.join(root, file)
            img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
            if img is None:
                continue
            img = cv2.resize(img, (256, 256))
            hist = extract_lbp_features(img)
            dist = chi2_distance(query_hist, hist)
            scores.append((dist, file, img))
scores.sort(key=lambda x: x[0])
top_matches = scores[:3]
plt.figure(figsize=(12, 6))
plt.subplot(1, 4, 1)
plt.imshow(query_img, cmap='gray')
plt.title("Query Image")
plt.axis('off')
```

```python
top_matches = scores[:3]
plt.figure(figsize=(12, 6))
plt.subplot(1, 4, 1)
plt.imshow(query_img, cmap='gray')
plt.title("Query Image")
plt.axis('off')
for i, (dist, fname, img) in enumerate(top_matches):
    plt.subplot(1, 4, i + 2)
    plt.imshow(img, cmap='gray')
    plt.title(f"Match {i+1}\nDist: {dist:.2f}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```



Case Study 1: Fashion E-Commerce Image Search

An online fashion retailer wants to improve customer experience by enabling customers to upload a photo of a clothing item they like (say, a shirt or a dress). The system should automatically find visually similar products from their catalog, helping customers find exactly what they want or discover similar styles.

Objective:

Use Content-Based Image Retrieval (CBIR) with color histograms to match the uploaded query image with items from the retailer's product database.

```python
# Case Study 1: Fashion E-Commerce Image Search
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity
import os
dataset_path = "D:/23108106/DEEPFASHION/img/img"
query_path = "D:/23108106/A33.jpg"
def extract_color_histogram(image, bins=(8, 8, 8)):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    hist = cv2.calcHist([hsv], [0, 1, 2], None, bins, [0,180, 0,256, 0,256])
    cv2.normalize(hist, hist)
    return hist.flatten()
def extract_gray_histogram(image, bins=256):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    hist = cv2.calcHist([gray], [0], None, [bins], [0, 256])
    cv2.normalize(hist, hist)
    return hist.flatten()
query_img = cv2.imread(query_path)
if query_img is None:
    raise ValueError("Query image not found!")
query_features_fashion = extract_color_histogram(query_img)
query_features_medical = extract_gray_histogram(query_img)
dataset_imgs = []
dataset_fashion_features = []
dataset_medical_features = []
max_images = 150
count = 0
for root, dirs, files in os.walk(dataset_path):
    for filename in files:
        if filename.lower().endswith(('.jpg', '.jpeg', '.png')):
            img_path = os.path.join(root, filename)
            img = cv2.imread(img_path)
            if img is None:
                continue
            dataset_imgs.append(img)
            dataset_fashion_features.append(extract_color_histogram(img))
```

```python
            dataset_fashion_features.append(extract_color_histogram(img))
            dataset_medical_features.append(extract_gray_histogram(img))
            count += 1
            if count >= max_images:
                break
    if count >= max_images:
        break
mode = 'fashion'
if mode == 'fashion':
    query_features = query_features_fashion
    dataset_features = dataset_fashion_features
elif mode == 'medical':
    query_features = query_features_medical
    dataset_features = dataset_medical_features
similarities = [cosine_similarity([query_features], [feat])[0][0] for feat in dataset_features]
ranked_indices = sorted(range(len(similarities)), key=lambda i: similarities[i], reverse=True)

plt.figure(figsize=(5,5))
plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
plt.title('Query Image')
plt.axis('off')
plt.show()
top_k = 3
for i in range(min(top_k, len(dataset_imgs))):
    idx = ranked_indices[i]
    plt.figure(figsize=(5,5))
    plt.imshow(cv2.cvtColor(dataset_imgs[idx], cv2.COLOR_BGR2RGB))
    plt.title(f"Similar Image {i+1} (Score: {similarities[idx]:.2f})")
    plt.axis('off')
    plt.show()
```

Query Image

Similar Image 1 (Score: 0.06)

Case Study 2: Medical Imaging Systems

Radiologists want to speed up diagnosis by searching for past medical scans (X-rays, MRIs) similar to a current patient's scan. The CBIR system extracts texture and color features from the scans and retrieves similar historical cases to assist in diagnosis and treatment planning.

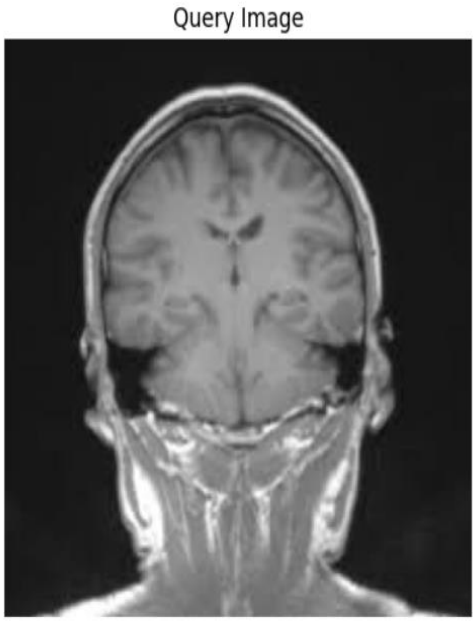
Objective:

Implement CBIR using grayscale image histograms to find the most similar X-ray or MRI images from a dataset, helping doctors in clinical decision making.

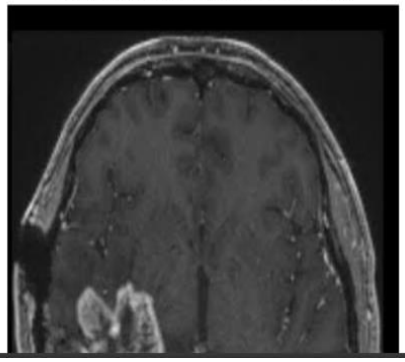
```
[5]: # Case Study 2: Medical Imaging Systems
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity
import os
dataset_path = "D:/23108106/BRAIN TUMOR MRI"
query_path = "D:/23108106/A20.jpg"
def extract_color_histogram(image, bins=(8, 8, 8)):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    hist = cv2.calcHist([hsv], [0, 1, 2], None, bins, [0,180, 0,256, 0,256])
    cv2.normalize(hist, hist)
    return hist.flatten()
def extract_gray_histogram(image, bins=256):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    hist = cv2.calcHist([gray], [0], None, [bins], [0, 256])
    cv2.normalize(hist, hist)
    return hist.flatten()
query_img = cv2.imread(query_path)
if query_img is None:
    raise ValueError("Query image not found!")
query_features_fashion = extract_color_histogram(query_img)
query_features_medical = extract_gray_histogram(query_img)
dataset_imgs = []
dataset_fashion_features = []
dataset_medical_features = []
max_images = 150
count = 0
for root, dirs, files in os.walk(dataset_path):
    for filename in files:
        if filename.lower().endswith(('.jpg', '.jpeg', '.png')):
            img_path = os.path.join(root, filename)
            img = cv2.imread(img_path)
            if img is None:
                continue
            dataset_imgs.append(img)
            dataset_fashion_features.append(extract_color_histogram(img))
            dataset_medical_features.append(extract_gray_histogram(img))
```

```
            dataset_medical_features.append(extract_gray_histogram(img))
            count += 1
            if count >= max_images:
                break
    if count >= max_images:
        break
mode = 'medical'
if mode == 'fashion':
    query_features = query_features_fashion
    dataset_features = dataset_fashion_features
elif mode == 'medical':
    query_features = query_features_medical
    dataset_features = dataset_medical_features
similarities = [cosine_similarity([query_features], [feat])[0][0] for feat in dataset_features]
ranked_indices = sorted(range(len(similarities)), key=lambda i: similarities[i], reverse=True)
plt.figure(figsize=(5,5))
plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
plt.title('Query Image')
plt.axis('off')
plt.show()
top_k = 3
for i in range(min(top_k, len(dataset_imgs))):
    idx = ranked_indices[i]
    plt.figure(figsize=(5,5))
    plt.imshow(cv2.cvtColor(dataset_imgs[idx], cv2.COLOR_BGR2RGB))
    plt.title(f"Similar Image {i+1} (Score: {similarities[idx]:.2f})")
    plt.axis('off')
    plt.show()
```

Query Image



Similar Image 1 (Score: 0.54)



Case Studies:

Case Study 1: Fashion Retail – Visual Product Search

Scenario: An online fashion retailer wants to allow users to upload images of clothing (like dresses, bags, or shoes) and find visually similar products in their inventory.

Objective: Implement a CBIR system that extracts color and texture features from a query image and retrieves the top visually similar items from a product catalog.

Features Used:

- **Color histogram in HSV space**
- **Local Binary Patterns (LBP) for texture**

Dataset Suggestion: Images of fashion products from the DeepFashion Dataset or Unsplash.

```
# Case Study 1: Fashion Retail - Visual Product Search
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from sklearn.metrics.pairwise import cosine_similarity
dataset_path = "D:/23108106/DEEPFASHION/img/img"
query_path = "D:/23108106/A33.jpg"
def feature(image):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    hist = cv2.calcHist([hsv],[0,1,2],None,[8,8,8],[0,180,0,256,0,256])
    cv2.normalize(hist,hist)
    return hist.flatten()
query = cv2.imread(query_path)
if query is None:
    raise ValueError("Query image not found!")
query = cv2.resize(query,(200,200))
query_feat = feature(query)
images, features = [], []
max_images = 150
count = 0
for root, _, files in os.walk(dataset_path):
    for file in files:
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            img = cv2.imread(os.path.join(root,file))
            if img is None:
                continue
            img = cv2.resize(img,(200,200))
            images.append(img)
            features.append(feature(img))
            count += 1
            if count >= max_images:
                break
    if count >= max_images:
        break
scores = [cosine_similarity([query_feat],[f])[0][0] for f in features]
idx = np.argsort(scores)[::-1][:3]
plt.imshow(cv2.cvtColor(query,cv2.COLOR_BGR2RGB))
```

```
plt.imshow(cv2.cvtColor(query,cv2.COLOR_BGR2RGB))
plt.title("Query")
plt.axis("off")
plt.show()
for i in idx:
    plt.imshow(cv2.cvtColor(images[i],cv2.COLOR_BGR2RGB))
    plt.title(f"Score: {scores[i]:.2f}")
    plt.axis("off")
    plt.show()
```

Query

Score: 0.06

Case Study 2: Medical Imaging – Case Matching for Diagnosis Support
Scenario: Radiologists use historical medical scans to assist diagnosis. A CBIR system can retrieve similar X-ray or MRI images from a database based on a new patient scan.
Objective: Implement CBIR using grayscale histogram and texture descriptors to find visually similar medical images.
Features Used:
- Grayscale histogram
- Texture descriptors like Gabor filters or LBP

Dataset Suggestion: NIH Chest X-ray Dataset, or MedPix®

```python
# Case Study 2: Medical Imaging - Case Matching for Diagnosis Support
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from sklearn.metrics.pairwise import cosine_similarity
dataset_path = dataset_path = "D:/23108106/MEDPIX/images/images"
query_path = "D:/23108106/A12.jpg"
def feature(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    hist = cv2.calcHist([gray],[0],None,[256],[0,256])
    cv2.normalize(hist,hist)
    return hist.flatten()
query = cv2.imread(query_path)
if query is None:
    raise ValueError("Query image not found!")
query = cv2.resize(query,(200,200))
query_feat = feature(query)
images, features = [], []
max_images = 150
count = 0
for root, _, files in os.walk(dataset_path):
    for file in files:
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            img = cv2.imread(os.path.join(root,file))
            if img is None:
                continue
            img = cv2.resize(img,(200,200))
            images.append(img)
            features.append(feature(img))
            count += 1
            if count >= max_images:
                break
    if count >= max_images:
        break
scores = [cosine_similarity([query_feat],[f])[0][0] for f in features]
idx = np.argsort(scores)[::-1][:3]
plt.imshow(cv2.cvtColor(query,cv2.COLOR_BGR2RGB))
```

```python
plt.title("Query")
plt.axis("off")
plt.show()
for i in idx:
    plt.imshow(cv2.cvtColor(images[i],cv2.COLOR_BGR2RGB))
    plt.title(f"Score: {scores[i]:.2f}")
    plt.axis("off")
    plt.show()
```

Query

Score: 0.80

Case Study 3: Wildlife Monitoring – Animal Identification from Camera Traps
Scenario: Wildlife researchers want to identify animals captured in camera trap images by comparing them to a database of known species images.
Objective: Implement a CBIR system to identify the most visually similar species from a query image.
Features Used:
□ Color histogram
□ Shape descriptors (e.g., Hu Moments)
Dataset Suggestion: Snapshot Serengeti Dataset

```python
# Case Study 3: Wildlife Monitoring - Animal Identification from Camera Traps
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from sklearn.metrics.pairwise import cosine_similarity
dataset_path = "D:/23108106/SERENGETI/test"
query_path = "D:/23108106/A31.jpg"
def feature(image):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    hist = cv2.calcHist([hsv],[0,1,2],None,[8,8,8],[0,180,0,256,0,256])
    cv2.normalize(hist,hist)
    return hist.flatten()
query = cv2.imread(query_path)
if query is None:
    raise ValueError("Query image not found!")
query = cv2.resize(query,(200,200))
query_feat = feature(query)
images, features = [], []
max_images = 150
count = 0
for root, _, files in os.walk(dataset_path):
    for file in files:
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            img = cv2.imread(os.path.join(root,file))
            if img is None:
                continue
            img = cv2.resize(img,(200,200))
            images.append(img)
            features.append(feature(img))
            count += 1
            if count >= max_images:
                break
    if count >= max_images:
        break
scores = [cosine_similarity([query_feat],[f])[0][0] for f in features]
idx = np.argsort(scores)[::-1][:3]
plt.imshow(cv2.cvtColor(query,cv2.COLOR_BGR2RGB))
```

```python
plt.imshow(cv2.cvtColor(query,cv2.COLOR_BGR2RGB))
plt.title("Query")
plt.axis("off")
plt.show()
for i in idx:
    plt.imshow(cv2.cvtColor(images[i],cv2.COLOR_BGR2RGB))
    plt.title(f"Score: {scores[i]:.2f}")
    plt.axis("off")
    plt.show()
```

Query

Score: 0.71

Case Study 4: Art Museum – Artwork Recommendation System
Scenario: A museum visitor takes a picture of a painting they like. The CBIR system suggests other artworks with similar style, color, or texture.
Objective: Build a system to retrieve artworks from a collection that are visually similar to the user's input.
Features Used:
□ Color histograms

□ Edge-based features (e.g., Canny, HOG)
Dataset Suggestion: WikiArt or Kaggle's Painter by Numbers dataset

```
# Case Study 4: Art Museum - Artwork Recommendation System
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from sklearn.metrics.pairwise import cosine_similarity
dataset_path = "D:/23108106/PAINTER BY NUMBERS"
query_path = "D:/23108106/A34.jpg"
def feature(image):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    hist = cv2.calcHist([hsv],[0,1,2],None,[8,8,8],[0,180,0,256,0,256])
    cv2.normalize(hist,hist)
    return hist.flatten()
query = cv2.imread(query_path)
if query is None:
    raise ValueError("Query image not found!")
query = cv2.resize(query,(200,200))
query_feat = feature(query)
images, features = [], []
max_images = 150
count = 0
for root, _, files in os.walk(dataset_path):
    for file in files:
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            img_path = os.path.join(root,file)
            img = cv2.imread(img_path)
            if img is None:
                continue
            img = cv2.resize(img,(200,200))
            images.append(img)
            features.append(feature(img))
            count += 1
            if count >= max_images:
                break
    if count >= max_images:
        break
scores = [cosine_similarity([query_feat],[f])[0][0] for f in features]
idx = np.argsort(scores)[::-1][:3]
```

```
scores = [cosine_similarity([query_feat],[f])[0][0] for f in features]
idx = np.argsort(scores)[::-1][:3]
plt.imshow(cv2.cvtColor(query,cv2.COLOR_BGR2RGB))
plt.title("Query")
plt.axis("off")
plt.show()
for i in idx:
    plt.imshow(cv2.cvtColor(images[i],cv2.COLOR_BGR2RGB))
    plt.title(f"Score: {scores[i]:.2f}")
    plt.axis("off")
    plt.show()
```

Query

Score: 0.10

Case Study 5: Industrial Quality Inspection – Detecting Defective Products
Scenario: In a manufacturing plant, a camera captures images of newly produced items. A CBIR system compares them to images of defect-free items to detect anomalies.
Objective: Retrieve reference images similar to a sample. A low similarity score may indicate a defect.
Features Used:
□ Gabor filters or texture patterns
□ Grayscale histogram

Dataset Suggestion: Create a sample dataset of "defect-free" and "defective" tile or metal surface images, or use industrial texture datasets from the KTH-TIPS dataset.

```python
# Case Study 5: Industrial Quality Inspection - Detecting Defective Products
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from sklearn.metrics.pairwise import cosine_similarity
dataset_path = "D:/23108106/KTH TIPS-2/KTH_TIPS"
query_path = "D:/23108106/A21.jpg"
def feature(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    hist = cv2.calcHist([gray],[0],None,[256],[0,256])
    cv2.normalize(hist,hist)
    return hist.flatten()
query = cv2.imread(query_path)
if query is None:
    raise ValueError("Query image not found!")
query = cv2.resize(query,(200,200))
query_feat = feature(query)
images, features = [], []
max_images = 150
count = 0
for root, _, files in os.walk(dataset_path):
    for file in files:
        if file.lower().endswith('.jpg', '.jpeg', '.png'):
            img_path = os.path.join(root,file)
            img = cv2.imread(img_path)
            if img is None:
                continue
            img = cv2.resize(img,(200,200))
            images.append(img)
            features.append(feature(img))
            count += 1
            if count >= max_images:
                break
    if count >= max_images:
        break
scores = [cosine_similarity([query_feat],[f])[0][0] for f in features]
idx = np.argsort(scores)[::-1][:3]
```

```python
scores = [cosine_similarity([query_feat],[f])[0][0] for f in features]
idx = np.argsort(scores)[::-1][:3]
plt.imshow(cv2.cvtColor(query,cv2.COLOR_BGR2RGB))
plt.title("Query")
plt.axis("off")
plt.show()
for i in idx:
    plt.imshow(cv2.cvtColor(images[i],cv2.COLOR_BGR2RGB))
    plt.title(f"Score: {scores[i]:.2f}")
    plt.axis("off")
    plt.show()
```

Query

Score: 0.91

https://github.com/Ayesha-Munir-Chohan/